

Intent-Based Networking with a Threat-Informed AI Gateway

How **ibn-core** turns a natural-language customer intent into a fulfilled network service under RFC 9315 — and how its inline AI-Gateway *rejects* unsafe requests before they ever reach the model or the network. End-to-end behaviour is captured as OpenTelemetry traces.

[RFC 9315 §5 Intent Handling](#)

[TMF921 v5.0.0 Intent API](#)

[OpenTelemetry / OTLP](#)

[OWASP LLM01](#)

[MCP RBAC + DLP \(arXiv 2511.20920\)](#)

[↔ Red-blue view](#)

[↔ ODA Canvas implementation](#)

[↓ Download \(HTML\)](#)

[Apache 2.0 · Vpnet Cloud Solutions](#)

1 · Executive summary

One platform; one trace format; two opposite outcomes — fulfilment and principled refusal.

ibn-core is the open-core of an AI-native intent platform. A customer expresses an intent in plain language (“I need internet for working from home”); the platform translates it with an LLM, orchestrates fulfilment across back-office systems via the Model Context Protocol (MCP), and reports compliance — all mapped to the six phases of **RFC 9315 §5** and the **TMF921** Intent API.

Every request first passes an inline **AI Gateway** with three deny-capable controls. The same request path therefore produces four characteristic outcomes, each emitted as an OpenTelemetry trace with GenAI, RFC 9315-phase and AI-Gateway-event attributes:

27

spans across 4 trace trees, loaded into Neo4j as a queryable graph

3

independent gateway controls, each able to halt the cycle

2

distinct refusal modes —

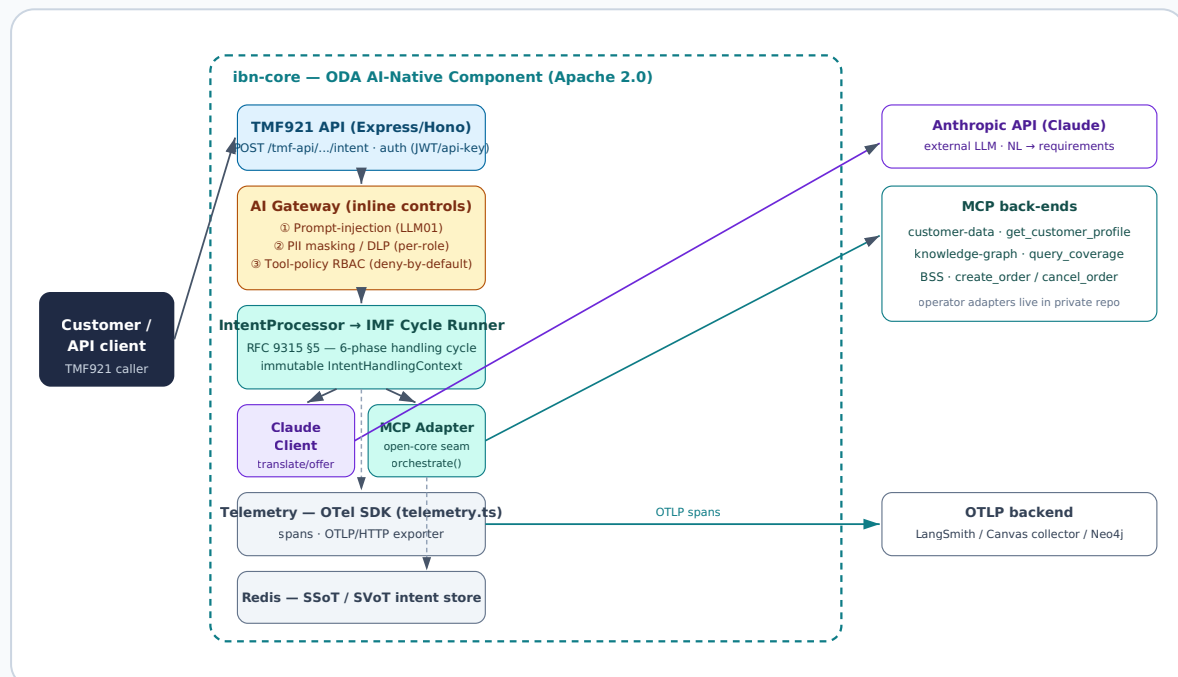
compliantFailure vs blocked

Scenario	What happens	Gateway verdict	Root	reportState	HTTP
A · happy path	Full lifecycle: ingest → translate → orchestrate (3×MCP + offer) → monitor → assess	all pass	OK	fulfilled	201
B · DLP redaction	Happy path, but PII fields in the profile are redacted before reaching the model	redact	OK	fulfilled	201
C · tool-policy deny	A customer - role caller tries an admin-only tool; orchestration is refused	deny (RBAC)	ERROR	compliantFailure	403
D · injection block	A jailbreak phrase is detected at ingest; the request is rejected before translation	block (LLM01)	ERROR	—	400

The key message: **a refusal is a first-class, observable outcome**. The platform does not silently fail or quietly “best-effort” around a policy — it stops at a named phase, emits a structured gateway event, and reports an auditable state.

2 · Architecture (C4 — container view)

System context: a Customer/API client drives ibn-core, which fans out to an LLM, three MCP back-ends and a Redis source-of-truth, while every component emits OTel spans to an OTLP backend.



C4 container view. Teal dashed = telemetry path. The MCP Adapter is the open-core seam: the interface is public, operator-specific implementations are private.

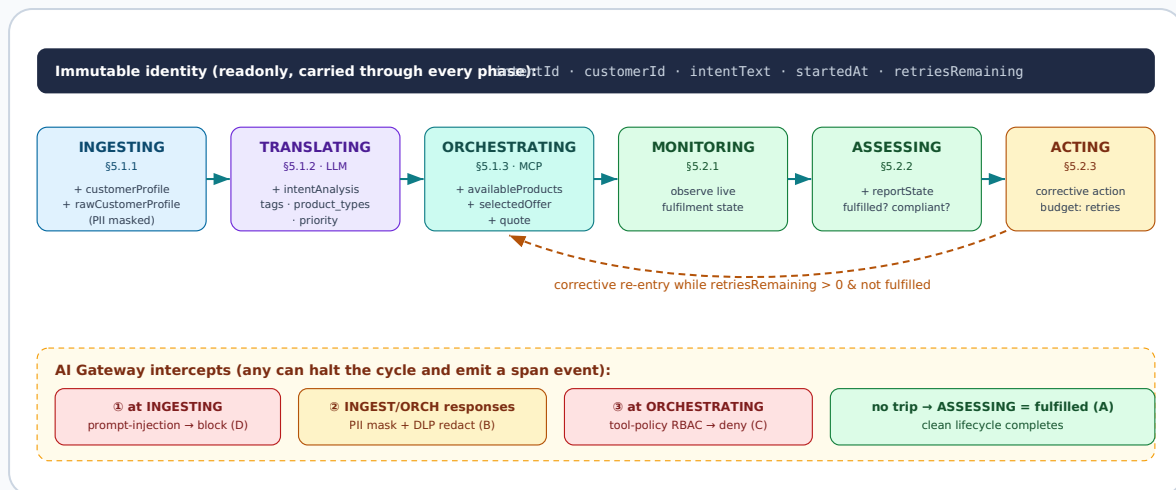
3 · The intent-handling cycle & how context is managed

RFC 9315 §5 defines six phases. ibn-core models them as

`IntentHandlingCycleRunner` phases over a single `IntentHandlingContext`.

Context discipline. `IntentHandlingContext` is *immutable*. Each phase receives the previous context and returns an *enriched copy* (object spread) — it is never mutated in place. Identity fields (`intentId` , `customerId` , `intentText` , `startedAt`) are *readonly* and carried unchanged end-to-end; each phase only

adds the fields it is responsible for. This makes the state evolution explicit, the trajectory replayable, and a failure attributable to the exact phase that produced it.



RFC 9315 §5 phases over an immutable, accumulating context. The amber band shows where each AI-Gateway control sits relative to the cycle.

4 · The AI Gateway — how the “reject” decision is made

Three independent, deny-capable controls. Each is rooted in a published control set and each emits a structured `ai_gateway.*` span event when it fires.

① Prompt-injection **block**

`prompt-injection-detection.ts` · OWASP LLM01

A signature engine matches the raw intent text against tiered regex patterns:

- **HIGH** — jailbreaks: `ignore previous instructions`, `pretend you`

② PII mask + DLP **redact**

`pii-masking.ts` + `DLPPolicy.ts` · MCP §4.4

Two layers protect data flowing to/from the model:

- **Input**: the customer profile is PII-masked before any external AI call.
- **Output**: DLP applies *per-tool, per-role field allowlists* to each

③ Tool-policy RBAC **deny**

`ToolPolicyEngine.ts` · MCP §4.1 · RFC 9315 P5

Deny-by-default allowlists per role; the role is resolved from the caller's API-key name:

- `customer` → `get_customer_profile`, `get_intent_status`, `get_quote`, `list_products`
- `agent` → + `catalog`, `quote`, `submit_order`, KG tools

are... , reveal
your system
prompt

- **MEDIUM** —
developer/
admin mode ,
bypass
safety ,
<script>
- **LOW** — drop
table , sudo ,
template/var
injection

Rule: HIGH always
blocks
(blockHigh=true);
MEDIUM/LOW are
configurable. A block
returns *before translation*
— the model never sees
the payload. →
Scenario D.

MCP response; non-
allowed fields become
[DLP_REDACTED]
(schema shape
preserved).

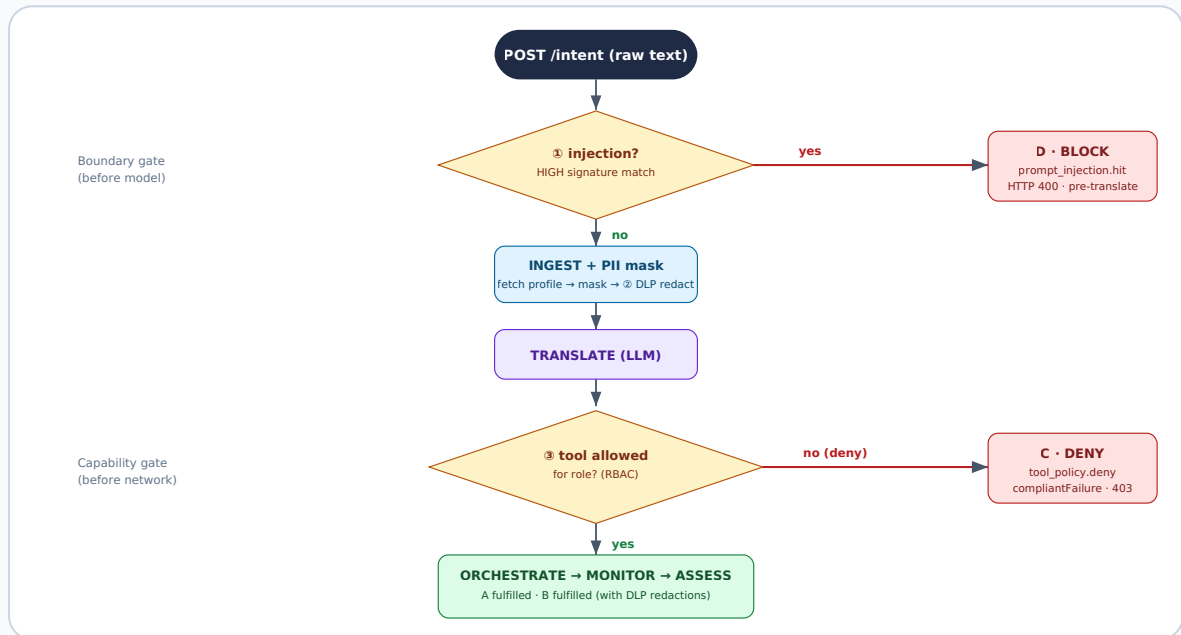
Rule: e.g. on
get_customer_profile
a customer sees only
non-identifying fields; raw
email / phone are
redacted. The cycle
continues — this is
containment, not refusal. →
Scenario B.

- admin → +
cancel_order ,
update_customer_profile,
manage_api_keys...

Rule: a tool not on the role's
allowlist is denied. A
customer invoking the
admin-only cancel_order
is refused at orchestration. →
Scenario C.

Two refusal semantics, deliberately different. A *policy* refusal (C) is a **compliantFailure**: the request was well-formed and understood, but the principal is not authorised — the system did the *right* thing by refusing, and records an auditable assessment (HTTP 403). A *security* block (D) is a hard reject at the boundary (HTTP 400): the input is hostile, so it is stopped before it can influence the model — there is no **reportState** because no intent was ever admitted.

4.1 · Decision logic (where each path forks)



The reject logic. Two diamonds, two hard exits. ① is a boundary gate (input never reaches the LLM); ③ is a capability gate (the action never reaches the network). ② (DLP) does not fork — it redacts in-line and the request proceeds.

5 · Inside the LLM step — what gets translated & what drives token cost

ibn-core makes **two** LLM calls (`claude-client.ts`), both single-turn JSON extractions at `temperature / max_tokens` fixed by config. Neither call ever sees raw PII — gate ② masks the profile first.



The two prompts, by composition. The fixed scaffold is constant; everything else is interpolated at runtime. Highlighted (amber) inputs in ② — the MCP catalog and bundle payloads — are the dominant input-token driver in production.

① `analyzeIntent(intent, maskedProfile)`

RFC 9315 §5.1.2 — TRANSLATING. The “understand” call.

In: a fixed extraction-instruction scaffold + the customer’s natural-language intent string + three masked profile fields (`segment` , `current_services` , `tenure`).

Out: structured JSON — `tags` (e.g. `work_from_home`), `product_types` (e.g. `broadband`), `priority` (e.g. `speed`, `reliability`). This is the literal “translation” of NL into network-actionable requirements.

② `generateOffer({intent, products, bundles, segment})`

RFC 9315 §5.1.3 — inside ORCHESTRATING, after the MCP fetches. The “decide” call.

In: a fixed offer-instruction scaffold + the `intentAnalysis` from ① + the **BSS product catalog** + the **KG bundle list** + the customer segment.

Out: JSON — `selected_products` , `bundle_recommendation` , pricing strategy, promotional `discounts` . This becomes the quote in the TMF921 response.

5.1 · What drives token usage

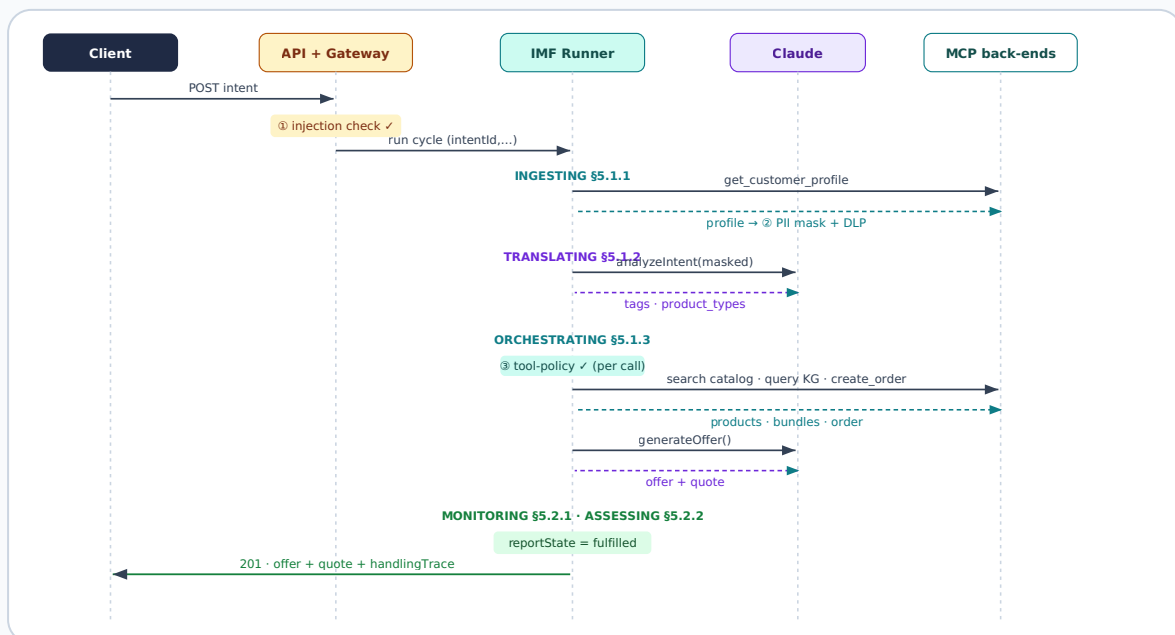
- **Input = fixed scaffold + interpolated runtime data**, and the runtime data dominates. For ① it’s the intent length + the masked profile; for ② it’s the **MCP catalog and bundle payloads** — usually the single largest contributor. A bigger product catalog ⇒ more input tokens.
- **Output = the size of the returned JSON**. More tags, more selected products, more discount lines ⇒ more output tokens.
- **Hard ceiling:** `max_tokens` in `ClaudeConfig` caps output; `temperature` and `model` are fixed per deployment.

- **Masking shifts the count slightly:** PII/DLP replace raw values with shorter sentinels *before* the prompt is built — e.g. Scenario B's translate input is 298 tok vs A's 312.
- **The gateway is the biggest lever of all.** A call only happens if the request gets that far: **D** (blocked at ingest) = **0** LLM calls; **C** (denied before the offer) = **1**; **A** / **B** = **2**. Token spend scales with how far a request is allowed to travel — *a refusal is also a cost saving*.

| Call | Phase | Dominant input drivers | in / out (A) |
|--------------------|-----------------------|---|--------------|
| ①
analyzeIntent | §5.1.2
translate | scaffold + intent text + masked profile | 312 / 118 |
| ②
generateOffer | §5.1.3
orchestrate | scaffold + analysis + catalog + bundles + segment | 96 / 54 |

Note: in the sample traces ② shows a smaller input than ① — these are illustrative figures. In production ② is typically the *larger* input call because it carries the full MCP catalog and bundle payloads; the highlighted bars in the diagram mark that real-world driver.

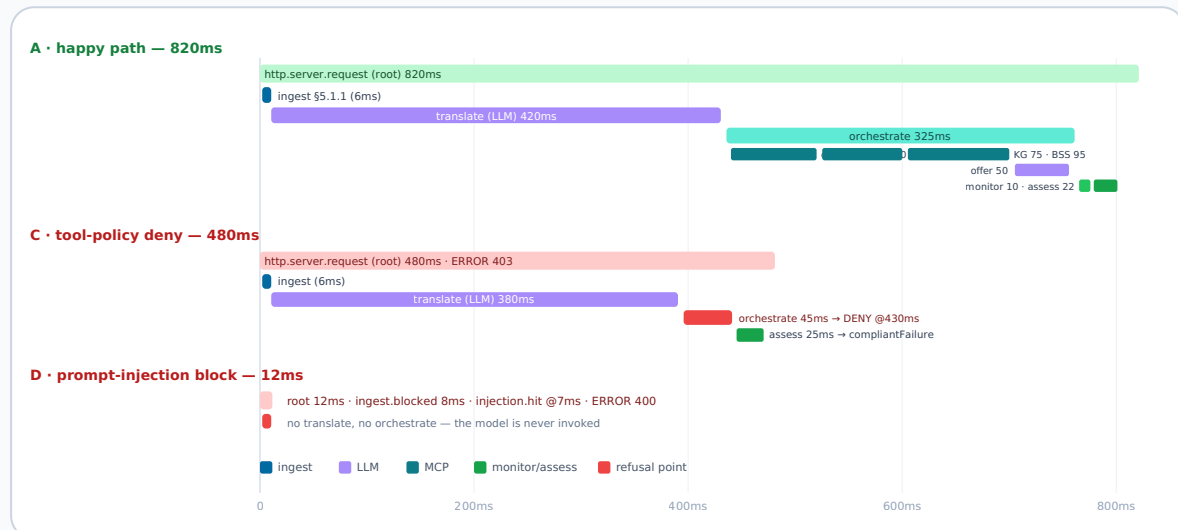
6 · End-to-end sequence (happy path, Scenario A)



Scenario A. Solid = request, dashed teal/purple = response. Gateway checkpoints (amber/teal mini-bars) sit inline: ① at entry, ② on the profile response, ③ before each MCP call.

7 · Timing — fulfilment vs. early refusal

Span durations are taken directly from the OTLP sample traces (root span = whole request). Refusals are dramatically cheaper than fulfilment — the gateway pays off precisely because it exits early.



Wall-clock, drawn to scale. The LLM translate call dominates fulfilment cost (~420ms). The injection block (D) ends in ~12ms — two orders of magnitude cheaper — because it never reaches the model.

| Scenario | Total | LLM calls | Input tok | Output tok | MCP calls | Refusal point |
|----------------------|-------|-----------|-----------|------------|-----------|----------------------|
| A · happy path | 820ms | 2 | 408 | 172 | 3 | — |
| B · DLP redaction | 820ms | 2 | 394 | 175 | 3 | — (redacted in-line) |
| C · tool-policy deny | 480ms | 1 | 305 | 88 | 0 | ORCHESTRATING (RBAC) |
| D · injection block | 12ms | 0 | 0 | 0 | 0 | INGESTING (boundary) |

8 · Evidence & reproducibility

OTLP trace artefacts

Four replayable OTLP/HTTP JSON files (the canonical reference payloads):

- `uc006-sample-trace-A-happy-path.json` — 10 spans, OK
- `uc006-sample-trace-B-dlp-redaction.json` — 10 spans, 2 DLP events
- `uc006-sample-trace-C-tool-policy-deny.json` — 5 spans, ERROR
- `uc006-sample-trace-D-prompt-injection-block.json` — 2 spans, ERROR

Resource attrs:

```
service.name=ibn-core ,
service.version=2.3.0 ,
oda.component.* . Span
families carry
  rfc9315.phase ,
  gen_ai.* , mcp.* ,
  tmf921.reportState ;
gateway events carry
  ai_gateway.* .
```

Loaded into Neo4j as a graph

1 service · 4 traces · 27 spans · 23 `CHILD_OF` · 4 gateway events.

```
// all gateway security events, by scenario
MATCH (s:0telSpan)-[:HAS_EVENT]->(e:0telEvent {dataset:
MATCH (s)-[:IN_TRACE]->(t:0telTrace)
RETURN t.scenario, e.name, s.name, e{.*} AS payload
ORDER BY t.scenario;
```

→ B: 2× `dlp.hit` on `intent.ingest` · C: `tool_policy.deny` on `intent.orchestrate` · D: `prompt_injection.hit` on `intent.ingest.blocked` .

Provenance / fidelity notes. (1) The sample traces are illustrative of the *Phase 1* trace shape — today the live service emits only auto-instrumentation spans; the domain/gateway spans shown here are emitted by the scenario harness and

represent what Phase 2/3 will produce from the real `claude-client.ts`, `MCPAdapter` and policy modules. (2) The `ai_gateway.*` rule identifiers in the samples (e.g. `PI-017`, `jailbreak-v2`, `DLP-EMAIL-001`) are demo labels; the real mechanisms are the regex signature engine, the per-role DLP field allowlists, and the RBAC allowlists described in §4. (3) On the primary cycle path the requester `SessionContext` is not yet threaded into MCP calls (tracked as a follow-up) — so live provenance attribution is pending that fix.

ibn-core — RFC 9315 Intent-Based Networking, ODA Canvas UC006 Custom Observability.

Standards: RFC 9315 (IRTF NMRG), TMF921 v5.0.0, OpenTelemetry, OWASP LLM01, MCP security controls (Errico, Ngiam & Sojan, arXiv 2511.20920). © 2026 Vpnet Cloud Solutions Sdn. Bhd. —

Apache License 2.0. This is a standalone document; all diagrams are inline SVG (no external assets).